

Republic of the Philippines
Laguna State Polytechnic University
Province of Laguna
College of Computer Studies

ITEC 204 – Data Structures and Algorithms

Name: Ryan Brando T. Si	Date: September 23, 2026
Section: BSIT-2B	Instructor:

I. Objective: Write a short explanation of what your program is designed to do. (Minimum of 3 sentences)

Answer:

This program is designed to act as a simple IT Automation Incident Ticket Manager that helps a support team keep track of tickets created whenever an automation bot fails. It lets the user add a new incident ticket, display all the tickets that are still active, search for one specific ticket using its Incident ID, and remove a ticket once it has been resolved. It also keeps count of how many tickets are still active at any given time, so the user always knows how many issues still need attention.

II. Program Operations: Briefly explain how your program performs each operation. (Insert Specific Code Line and Your Explanation)

Add Incident:

```
tickets.append(new_ticket)
```

The program asks the user to type in the Incident ID, Bot name, and Short Description, checks that the ID doesn't already exist in the list, then builds a dictionary out of those three values and appends it to the end of the `tickets` list.

Display Incidents:

```
for ticket in tickets: print(...)
```

The program loops through every item currently in the `tickets` list and prints each one's Incident ID, Bot, and Short Description in a simple table layout so the user can see every active ticket at once.

Search Incident:

```
if ticket["id"] == incident_id:
```

The program asks for an Incident ID, then loops through the list comparing each ticket's ID to the one entered. If it finds a match, it prints that ticket's full details; if it finishes looping without finding one, it prints a not-found message instead.

Remove Resolved Incident:

```
tickets.remove(ticket)
```

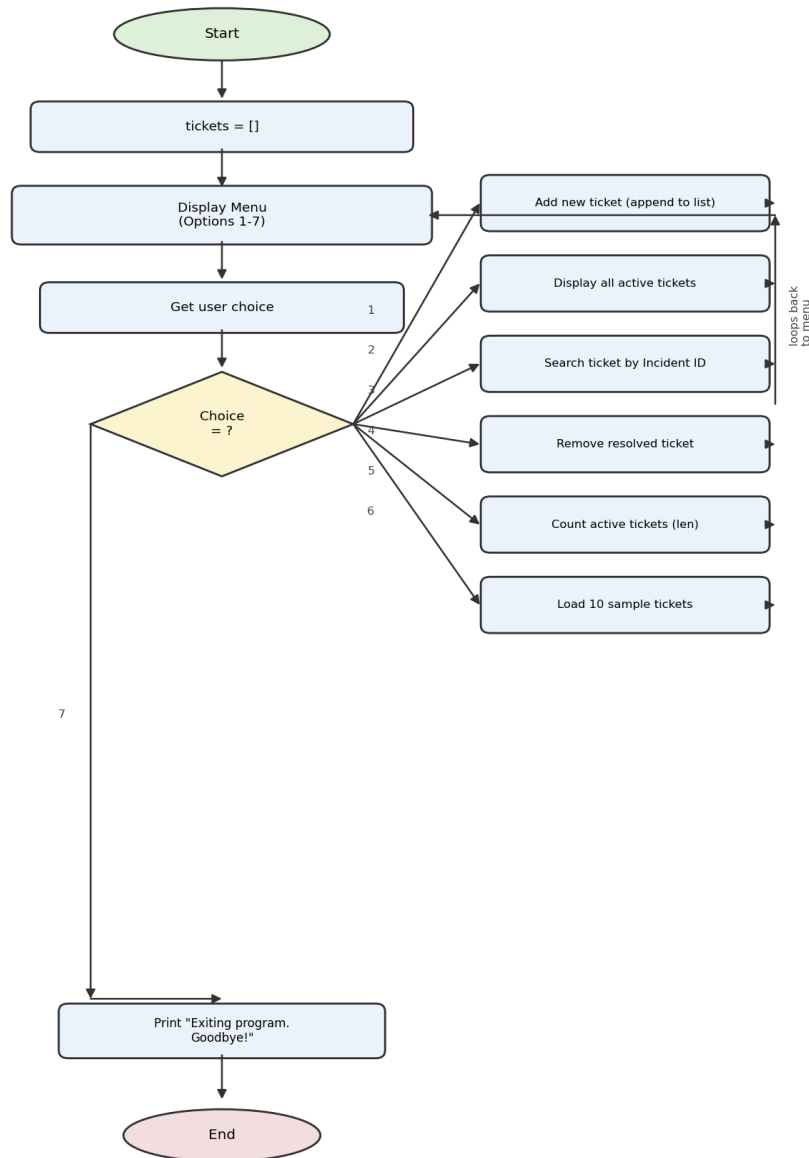
Similar to search, the program loops through the list looking for a matching Incident ID. Once it finds the ticket, it calls `tickets.remove(ticket)` to take that ticket out of the list completely, simulating the ticket being marked as resolved and closed.

Count Active Incidents:

```
len(tickets)
```

This one is the simplest — the program just uses Python's built-in `len()` function on the tickets list to instantly get and display how many tickets are currently active.

III. Flowchart: Create a flowchart showing the overall process and logic of your program.



IV. Conclusion: Briefly explain what you learned from the activity and how the linear data structure was applied.

Honestly, this activity looked scary at first because of all the GitHub folder stuff, but once I got the program itself working it wasn't that bad. Using a list for the tickets made sense once I thought about it — it's basically just a container I can keep adding to, loop through, and remove things from, which is exactly what a ticket system needs to do. The part I had to be careful with was the remove and search functions, since I had to make sure I was comparing the Incident ID correctly or it just wouldn't

find anything. I also learned that testing with the sample data first, instead of typing everything by hand every time, saved me a lot of time. Overall, I understood better how linear data structures like lists work in a real, practical scenario instead of just a textbook example, and I feel more confident now going into the next lab exercises.